CH-5 PART - III

In-Depth Study Notes: Oracle SQL Pivot, Cross Tab, and Matrix Reports

As an expert Oracle SQL Developer, one of the most highly sought-after skills is the ability to transpose rows into columns. The execution log provided (CH5-3.TXT) focuses entirely on building **Pivot, Cross Tab, and Matrix Reports** using conditional aggregation. This technique eliminates the need to export data to Excel by allowing the Oracle Engine to dynamically format multi-dimensional data directly within the SELECT clause.
Below is the technical breakdown of these advanced architectural reporting techniques.

Concept 1: The Architecture of Matrix Reporting (Rows to Columns)

Core Purpose: Standard GROUP BY queries output data vertically (as rows). A Matrix or Pivot report transposes this data horizontally (as columns). This allows a business to view multiple aggregated dimensions (like the average salary for Departments 20, 60, and 110) side-by-side in a single row instead of reading down a list 1, 2.
Keyword Breakdown:

- **GROUP BY vs. Conditional Aggregation**: Instead of using GROUP BY DEPARTMENT_ID (which returns multiple rows) 1, we omit the GROUP BY clause entirely. We force the transposition by wrapping a conditional statement (CASE or DECODE) directly *inside* an aggregate function (SUM, AVG, COUNT) for each distinct column we want to create 3, 4.

Syntax Breakdown:

```
-- Standard Vertical Grouping
SELECT group_col, SUM(target_col) FROM table_name GROUP BY group_col;

-- Horizontal Matrix / Pivot Reporting
SELECT SUM(conditional_logic_for_group_1) AS G1_SUM,
       SUM(conditional_logic_for_group_2) AS G2_SUM
FROM table_name;
```

Detailed Example: Instead of listing departments vertically, we want a single-row executive summary showing the average salary paid exclusively to Departments 20, 80, and 110 side-by-side 2, 5:

```
SELECT AVG(CASE WHEN DEPARTMENT_ID = 20 THEN SALARY END)
DEPT_20_AVG_SAL,
       AVG(CASE WHEN DEPARTMENT_ID = 80 THEN SALARY END)
DEPT_80_AVG_SAL,
       AVG(CASE WHEN DEPARTMENT_ID = 110 THEN SALARY END)
DEPT_110_AVG_SAL
FROM EMPLOYEES;
```

Execution Logic:

1. **Row-by-Row Evaluation**: The engine reads the first row. If the employee is in Department 20, the first CASE statement returns their SALARY. The other two CASE statements evaluate to NULL (since there is no ELSE clause) 3.
2. **Aggregation Phase**: The AVG() function sweeps down each newly generated column. Because aggregate functions intrinsically ignore NULL values, the DEPT_20_AVG_SAL column only averages the salaries of actual Department 20 employees 3.
3. **Projection**: A single row containing the three distinct average buckets is returned 3, 4.

Pro-Tips/Best Practices:

- **Syntax Strictness**: When attempting to filter multiple departments in a standard WHERE clause, you must use the IN operator (e.g., WHERE DEPARTMENT_ID IN (20, 80, 110)). Using shorthand like WHERE (20, 80, 110) triggers an **ORA-00920: invalid relational operator** error 5.

Concept 2: Matrix Reporting using DECODE

Core Purpose:DECODE is an Oracle-proprietary function that acts as an inline "IF-THEN-ELSE" statement. It is frequently used in matrix reporting because its syntax is significantly shorter and cleaner than an ANSI CASE expression when performing exact equality checks.

Keyword Breakdown:

- **DECODE(column, search_value, result_value)**: Checks if the *column* equals the *search_value*. If true, it outputs the *result_value*. If false, it defaults to NULL (which is perfect for aggregate exclusion).
- *Why use DECODE over CASE?* DECODE requires fewer keystrokes for simple equality matching (DECODE(DEPT_ID, 20, SALARY) vs CASE WHEN DEPT_ID = 20 THEN SALARY END). However, DECODE cannot handle ranges (like < or >), whereas CASE can 4, 6.

Syntax Breakdown:

```
SELECT aggregate_function(DECODE(eval_column, search_1,
target_column)) alias_1,
       aggregate_function(DECODE(eval_column, search_2,
target_column)) alias_2
FROM table_name;
```

Comparison of Matrix Conditional Logic:

Feature,DECODE,CASE
Standard,Oracle Proprietary,ANSI Standard
Logic Allowed,Exact equality matches only (=),"Inequalities (<, >), BETWEEN, IN, LIKE"
Verbosity,Highly concise,More verbose

Detailed Example: To build a matrix showing the absolute minimum salary paid under three different specific managers:

```
SELECT MIN(DECODE(MANAGER_ID, 100, SALARY)) MGR_100_MIN_SAL,
       MIN(DECODE(MANAGER_ID, 124, SALARY)) MGR_124_MIN_SAL,
       MIN(DECODE(MANAGER_ID, 149, SALARY)) MGR_149_MIN_SAL
FROM EMPLOYEES;
```

Execution Logic: For the first column, the engine checks every row. If MANAGER_ID is 100, it feeds the SALARY value to the MIN() function. If the manager is not 100, DECODE outputs NULL. The MIN() function calculates the lowest non-null value for that specific column expression 6.

Pro-Tips/Best Practices:

- **Counting with Constants**: If you need to COUNT rather than SUM, you do not need to return the salary. You can instruct the DECODE or CASE to return a literal string like '*' or 'OM NAMAH SHIVAYA'. Since COUNT just tallies the number of non-null instances, returning any random literal string will successfully increment the count for that matrix bucket 7, 8.

Concept 3: Advanced Function-Based Pivoting

Core Purpose: Matrix reporting is not limited to raw column data. You can embed deep string and mathematical functions inside the conditional logic to pivot data based on dates, string manipulation, or calculated numeric tiers.

Keyword Breakdown:

- **TO_CHAR(date, 'fmt')**: Used to extract specific date parts, such as Quarter ('Q'), Week of Year ('W'), or Year ('RR') to build temporal matrices 7, 9, 10.
- **MOD() / CEIL()**: Mathematical functions used to determine Odd/Even groupings or Halves of a whole 11.
- **LTRIM() / SUBSTR() / TRANSLATE()**: Advanced character manipulation to strip away consonants and evaluate vowels 12, 13.

Detailed Example 1 (Temporal Matrix): Finding the total salary paid in Q1, the highest salary in Q2, and the average salary in Q4:

```
SELECT SUM(DECODE(TO_CHAR(HIRE_DATE,'Q'), 1, SALARY)) HQ1_SUM_SAL,
       MAX(DECODE(TO_CHAR(HIRE_DATE,'Q'), 2, SALARY)) HQ2_MAX_SAL,
       AVG(DECODE(TO_CHAR(HIRE_DATE,'Q'), 4, SALARY)) HQ4_AVG_SAL
FROM EMPLOYEES;
```

Detailed Example 2 (Complex String Evaluation Pivot): To pivot a sum of salaries specifically for employees whose Last Name's *first vowel* is an 'A', you must aggressively strip all consonants using LTRIM, then grab the first remaining character using SUBSTR 13, 14:

```
SELECT SUM(CASE
SUBSTR(LTRIM(UPPER(LAST_NAME),'QWRTYPSDFGHJKLZXCVBNM'),1,1) WHEN 'A'
THEN SALARY END) LMVA_SUM_SAL
FROM EMPLOYEES;
```

Detailed Example 3 (Mathematical Pivot): To calculate the total salaries of employees hired in "Odd" numbered months (1, 3, 5, etc.) vs the "First Half" of the year 11:

```
-- Odd Months using MOD (remainder of division by 2)
SELECT SUM(CASE MOD(TO_CHAR(HIRE_DATE,'MM'), 2) WHEN 1 THEN SALARY
END) ODD_MONTH_SUM_SAL
FROM EMPLOYEES;

-- First Half of Year using CEIL (rounding up division by 6)
```

```
SELECT COUNT(CASE CEIL(TO_CHAR(HIRE_DATE,'MM') / 6) WHEN 1 THEN '*'
END) FH_OF_YEAR_CNT
FROM EMPLOYEES;
```

Execution Logic:

1. The inner-most functions (UPPER, TO_CHAR, LTRIM, MOD) manipulate the row's raw data into a new calculated string or number 9, 11, 12.
2. The CASE/DECODE evaluates this newly derived value against the search condition 12, 13.
3. The outer Aggregate function (SUM, MAX, COUNT) processes the result 11, 13.

Pro-Tips/Best Practices (Error Handling):

- **Missing Parentheses (ORA-00907)**: When nesting 3 or 4 functions deep inside a matrix column, missing a single closing parenthesis will instantly halt execution and throw an **ORA-00907: missing right parenthesis** error. Always format complex queries across multiple lines to track your brackets 9, 10, 15.
- **Datatype Consistency (ORA-00932)**: The THEN and ELSE outcomes inside a single CASE expression **must share the same data type**. If your aggregate function is MIN(), and your CASE statement returns a literal string '*' for TRUE but the aggregate expects a NUMBER based on other conditions, Oracle will throw an **ORA-00932: inconsistent datatypes: expected CHAR got NUMBER** error. Ensure you aren't mixing string literals and numeric column projections within the same aggregate evaluation 16.